

NumCompute-Stream: Technical Report

Assignment 2.2 — COMP 5004 Author: Saadman Sakib

1. Design Decisions

Package Architecture

In this assignment, the package of group Assignment 2.1 i.e. numcompute, was extended to numcompute_stream. Here, four additional modules were introduced which were the tree.py, ensemble.py, stream.py and visualise.py and their corresponding tests were implemented as test_ensemble.py, test_tree.py, test_stream.py, test_visualise.py. Some necessary updates were made to four pre-existing modules which included the preprocessing.py, stats.py, metrics.py and pipeline.py. A consistent API was followed for all the streaming components such that `partial_fit(X)` was demonstrated by transformers, `partial_fit(X,y)` was demonstrated by estimators and finally `update()`, `reset()` and `result()` were demonstrated by metric trackers. The main reason behind the consistency of streaming components was to resemble scikit-learn's learning component that operates online.

Decision Tree

DecisionTreeClassifier utilises Gini impurity and entropy for splitting as well as customizable `max_depth`, `min_samples_split` and `max_features` (int, float, 'sqrt', 'log2'). The tree is generated in a recursive manner utilising the midpoint threshold between values of unique features. Before the fitting and prediction process, NaN values are replaced with the corresponding column means. The `partial_fit()` technique utilises an accumulation method aided by a buffer where it adds each new chunk to the already-seen data and thus, the tree is again built from start. This method is much simpler and guaranteed for accuracy compared to the Hoeffding tree which we did not use. The Hoeffding tree is a true online algorithm that never sees previous data again. Our approach, therefore, guarantees accuracy with a tradeoff of using more memory as the stream keeps on growing.

Ensemble

EnsembleClassifier uses two methods such as `random_forest` (bootstrap sampling + `max_features = 'sqrt'` per tree) and `bagging` (bootstrap sampling + all features). The predictions use majority prioritisation among all `n_estimators` trees. `partial_fit()` gathers

all previously seen data and calls the `partial_fit()` method on each tree using a new bootstrap sample of the gathered dataset.

Streaming Statistics and Metrics

`StandardScaler.partial_fit()` utilises Welford's batch-based algorithm to sustain numerical stability for running mean and variance without having to store raw data. `StreamingMetrics` maintains an ongoing confusion matrix that incrementally grows larger when newer class labels come from later incoming chunks. A rolling window buffer holds the latest `window_size` chunks for localised metric evaluation. `StreamingStats`, in similar fashion, utilizes Welford's batch method and maintains a sliding-window histogram as an option.

Visualisation

All the plot functions take a parameter called 'show' which can be set to False. This ensures compatibility between interactive scripts and notebooks as in automated runs by suppressing pop-up windows. Another parameter `save_path` is added to write figures directly on the disk. The three relevant functions are `plot_metric_over_time`, `compare_models` and `plot_predictions_vs_ground_truth` with an additional `plot_confusion_matrix`. Accurate predictions are represented with the colour green and incorrect predictions are shown in red in the prediction plots.

2. Testing and Edge Cases

Test Suite

The test suite contains 294 unit tests across 13 files. Tests encompass both standard functionality and streaming-centric behaviour.

Key Edge Cases Tested

- NaN features in `DecisionTreeClassifier` are replaced with column means, prediction proceeds without error
- In case of Zero-variance features in `StandardScaler.partial_fit()`, standard deviation was set to 1.0 to prevent division by zero
- In case of empty inputs, `ValueError` is raised with descriptive message

- In case of shape mismatches between X and y, ValueError is raised
- In case of single-class input, root node becomes a leaf immediately
- In case of predict() before fit(), RuntimeError is raised
- In case of StreamingMetrics.result() before any update(), RuntimeError is raised
- In case of Rolling window in StreamingMetrics, only last window_size chunks contribute
- For incremental label expansion, confusion matrix resizes when new classes comes in
- For Pipeline.partial_fit() with final estimator, y is correctly passed to the last step regardless of the name of parameter

Debugging Evidence

During the process of development, two bugs were identified and corrected through tests:

(1) Pipeline.partial_fit() failed to pass y to the final estimator because _accepts_y() checks for a parameter named y but DecisionTreeClassifier.partial_fit uses y_chunk and this was fixed by always passing y to the final step.

(2) A RuntimeWarning for invalid division in F1 computation was handled using np.errstate(divide='ignore', invalid='ignore').

3. Benchmark Results

All benchmarks run on Python 3.14.4, NumPy 2.4.4, Windows, with n_repeat=2.

Benchmark 1: Decision Tree vs Random Forest (Streaming Fit)

Model	Fit time (s)	Accuracy
DecisionTree (depth=5)	0.6146	0.9300
RandomForest (10 trees)	0.7201	0.9250

Speedup factor (tree / forest): 0.85x

Therefore, Random Forest takes 1.2x longer to train than a single decision tree.

Benchmark 2: Impact of chunk size on streaming fit time

Chunk size	Number of Chunks	Fit time (s)
50	8	1.5290
100	4	0.8420
200	2	0.5081
500	1	0.3378
1000	1	0.3376

Smaller chunks cause more `partial_fit()` calls, each rebuilding the tree on a larger accumulated dataset, thus smaller chunks need more time for fitting.

Benchmark 3: Loop-based vs vectorised accuracy computation

Method	Time (s)
Python loop (n = 100,000)	0.007309
NumPy vectorised (n = 100,000)	0.000084

Speedup: 87.1x faster with vectorisation

Benchmark 4: Model memory footprint as chunks accumulate

Chunk	Samples seen	Memory (bytes)	Accuracy
1	200	17600	0.9600

2	400	35200	0.9350
---	-----	-------	--------

4. Reflections

What worked well

The modular design made the process of adding streaming support much straightforward where each module's `partial_fit()` could be added independently without breaking existing `fit()` and `transform()` behaviour. The `StreamTrainer` abstraction cleanly separates training logic from logging and metric tracking, allowing any compatible model to be plugged in. The `Pipeline.partial_fit()` chains preprocessing and model updates in a single call, closely mirroring the scikit-learn streaming API.

Limitations and trade-offs

The buffer-based `partial_fit()` for decision trees is not truly online as it stores all accumulated data and rebuilds the tree from scratch on each chunk. A true online algorithm (e.g. Hoeffding/VFDT) would be more memory-efficient for very large streams but would also be significantly complicated for correct implementation from scratch using NumPy. The current approach prioritises accuracy and simplicity within the constraints of the assignment at the cost of using more memory.

Numerical stability

Welford's batch algorithm for `StandardScaler.partial_fit()` was selected particularly to evade the cancellation that occurs with naive two-pass variance computation over chunks. This ensures numerically stable scaling across arbitrarily multiple chunks.

AI Use Acknowledgement

Generative AI (Claude, Anthropic) was used as an assistant for concept exploration and programming help in this assignment. All the codes, design decisions, and written content were first understood, reviewed by the author and only then were implemented and tested.

